Advanced Data Structures

Binary Search Trees, Hash Tables, and Python Generators

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

Move beyond linear data structures by introducing hierarchical structures (Trees) and revisit direct access structures (Hash Tables). We will delve into implementation and analysis of Binary Search Trees (BSTs), explore Python's generator mechanism ('yield'), and understand mechanics of Hash Tables for efficient lookups.

The Search Problem and Motivation

Revisiting the Efficiency of Search

We frequently need to implement the Map ADT (storing key-value pairs) and efficiently perform lookups.

Review of Linear Structures

- **Unsorted List/Array:** Insertion is $\mathcal{O}(1)$ (amortised if appending). Search requires a linear scan: $\mathcal{O}(N)$.
- **Sorted List/Array:** Search can use Binary Search: $\mathcal{O}(\log N)$. However, insertion requires shifting elements to maintain order: $\mathcal{O}(N)$.

The Trade-Off

We face a trade-off between fast insertion and fast search in linear structures.

Revisiting the Efficiency of Search

We frequently need to implement the Map ADT (storing key-value pairs) and efficiently perform lookups.

Review of Linear Structures

- **Unsorted List/Array:** Insertion is $\mathcal{O}(1)$ (amortised if appending). Search requires a linear scan: $\mathcal{O}(N)$.
- **Sorted List/Array:** Search can use Binary Search: $\mathcal{O}(\log N)$. However, insertion requires shifting elements to maintain order: $\mathcal{O}(N)$.

The Trade-Off

We face a trade-off between fast insertion and fast search in linear structures.

Revisiting the Efficiency of Search

The Trade-Off

We face a trade-off between fast insertion and fast search in linear structures.

The Goal

Can we design dynamic data structures that achieve better than linear time for both insertion and search?

- Goal 1: $\mathcal{O}(\log \mathrm{N})$ for Insert/Search/Delete (Trees).
- Goal 2: $\mathcal{O}(1)$ average time (Hashing).

Introduction to Trees

Trees: A Hierarchical Data Structure

Definition

A Tree is a hierarchical, non-linear data structure consisting of nodes connected by edges. It is defined recursively: a tree consists of a root node and zero or more subtrees, connected to the root by an edge.

Key Characteristics

- **Hierarchical:** Represents relationships (e.g., file systems, organizational charts).
- **Acyclic:** Contains no cycles.
- **Rooted:** Has a designated starting node.

Trees: A Hierarchical Data Structure

Definition

A Tree is a hierarchical, non-linear data structure consisting of nodes connected by edges. It is defined recursively: a tree consists of a root node and zero or more subtrees, connected to the root by an edge.

Key Characteristics

- **Hierarchical:** Represents relationships (e.g., file systems, organizational charts).
- **Acyclic:** Contains no cycles.
- **Rooted:** Has a designated starting node.

Tree Terminology

Root: The topmost node (no parent).

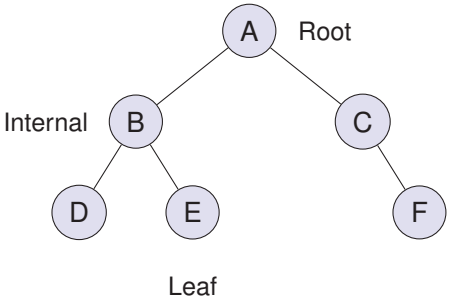
Parent/Child: Direct connection above/below.

Leaf: A node with no children.

Internal Node: A node with at least one child.

Depth: Length of the path from the root to the node.

Height (H): The maximum depth of any node.



The Importance of Height

The efficiency of most tree algorithms depends on the height (H) of the tree.

Tree Terminology

Root: The topmost node (no parent).

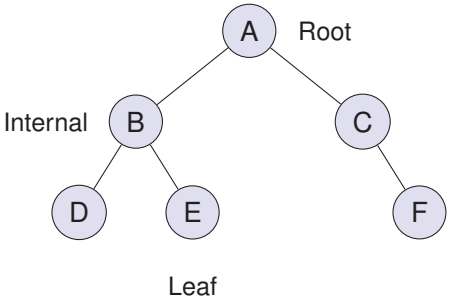
Parent/Child: Direct connection above/below.

Leaf: A node with no children.

Internal Node: A node with at least one child.

Depth: Length of the path from the root to the node.

Height (H): The maximum depth of any node.



The Importance of Height

The efficiency of most tree algorithms depends on the height (H) of the tree.

Binary Trees

Definition

A Binary Tree is a tree in which each node has **at most two children**, referred to as the ‘left child‘ and the ‘right child‘.

Properties Relating Height and Nodes

- A binary tree with N nodes has a minimum height of $\lfloor \log_2 N \rfloor$.
- If we can keep the tree balanced (height $\approx \log_2 N$), operations dependent on height will be efficient ($\mathcal{O}(\log N)$).

Binary Trees

Definition

A Binary Tree is a tree in which each node has **at most two children**, referred to as the ‘left child‘ and the ‘right child‘.

Properties Relating Height and Nodes

- A binary tree with N nodes has a minimum height of $\lfloor \log_2 \mathrm{N} \rfloor$.
- If we can keep the tree balanced (height $\approx \log_2 \mathrm{N}$), operations dependent on height will be efficient ($\mathcal{O}(\log \mathrm{N})$).

Binary Search Trees (BSTs)

The Binary Search Tree (BST) Invariant

Definition

A Binary Search Tree (BST) is a Binary Tree that satisfies the **BST Invariant**, enabling efficient searching.

The Binary Search Tree (BST) Invariant

Definition

A Binary Search Tree (BST) is a Binary Tree that satisfies the **BST Invariant**, enabling efficient searching.

The BST Invariant

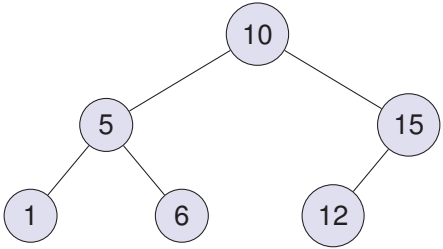
For any node N in the tree:

1. All keys in the left subtree of N are less than the key of N.
2. All keys in the right subtree of N are greater than the key of N.

The Binary Search Tree (BST) Invariant

Definition

A Binary Search Tree (BST) is a Binary Tree that satisfies the **BST Invariant**, enabling efficient searching.



BST Implementation: Structure (tree.py)

We implement the BST using two classes: ‘BinaryTree‘ (the container) and ‘TreeNode‘ (the recursive structure).

```
1  class BinaryTree:
2      def __init__(self):
3          self.root = None
4      # ... methods (insert, lookup, delete, walk) ...
5
6  class TreeNode:
7      def __init__(self, key, value):
8          self.key = key
9          self.value = value
10         self.left = None
11         self.right = None
12     # ... recursive methods (insert, find, delete, walk) ...
```

BST Implementation: Structure (tree.py)

We implement the BST using two classes: ‘BinaryTree‘ (the container) and ‘TreeNode‘ (the recursive structure).

Design Choice

Operations are implemented recursively on ‘TreeNode‘. The ‘BinaryTree‘ acts as a wrapper managing the ‘root‘.

BST Operation: Lookup (Find)

To find a key K, we exploit the BST invariant.

Algorithm

1. Start at the root.
2. If K equals the current key, return the node.
3. If K is less, recursively search the left subtree.
4. If K is greater, recursively search the right subtree.
5. If the subtree is empty ('None'), K is not found.

Complexity

The time complexity is proportional to the height of the tree, H. $T(N) = \mathcal{O}(H)$.

BST Operation: Lookup (Find)

To find a key K, we exploit the BST invariant.

Algorithm

1. Start at the root.
2. If K equals the current key, return the node.
3. If K is less, recursively search the left subtree.
4. If K is greater, recursively search the right subtree.
5. If the subtree is empty ('None'), K is not found.

Complexity

The time complexity is proportional to the height of the tree, H. $T(N) = \mathcal{O}(H)$.

BST Implementation: Lookup (Find) I

Listing 1: TreeNode.find (tree.py)

```
54      def find(self, key):
55          if key == self.key:
56              return self
57          if key < self.key:
58              # left
59              if self.left is None:
60                  return None
61              else:
62                  return self.left.find(key) # Recursive call
63          else:
64              # right
```

BST Implementation: Lookup (Find) II

```
65                 if self.right is None:
66                     return None
67                 else:
68                     return self.right.find(key) # Recursive call
```

BST Operation: Insertion

Insertion must maintain the BST invariant. New nodes are always inserted at a leaf position.

Algorithm

1. Perform a search for the key K.
2. The search will terminate at the ‘None‘ pointer where the new node should be attached.
3. Create the new node and attach it.

Complexity

Similar to search, insertion takes $\mathcal{O}(H)$ time.

BST Operation: Insertion

Insertion must maintain the BST invariant. New nodes are always inserted at a leaf position.

Algorithm

1. Perform a search for the key K.
2. The search will terminate at the ‘None‘ pointer where the new node should be attached.
3. Create the new node and attach it.

Complexity

Similar to search, insertion takes $\mathcal{O}(H)$ time.

BST Implementation: Insertion I

Listing 2: TreeNode.insert (tree.py)

```
36      def insert(self, key, value):
37          if key < self.key:
38              # left
39              if self.left is None:
40                  # Base Case: Found insertion point
41                  self.left = TreeNode(key, value)
42              else:
43                  # Recursive Step
44                  self.left.insert(key, value)
45          else:
```

BST Implementation: Insertion II

```
46                # right (Handles key >= self.key in this implementation
                      )
47                if self.right is None:
48                    self.right = TreeNode(key, value)
49                else:
50                    self.right.insert(key, value)
```

Tree Traversal

How do we visit every node exactly once?

Types of Depth-First Traversal

The order depends on when the current node (N) is visited relative to its left (L) and right (R) subtrees.

Pre-order: N, L, R (e.g., copying the tree structure).

In-order: L, N, R.

Post-order: L, R, N (e.g., deleting the tree structure).

In-order Traversal and BSTs

An in-order traversal of a BST visits the keys in ascending sorted order. This is a direct consequence of the BST invariant.

Tree Traversal

How do we visit every node exactly once?

Types of Depth-First Traversal

The order depends on when the current node (N) is visited relative to its left (L) and right (R) subtrees.

Pre-order: N, L, R (e.g., copying the tree structure).

In-order: L, N, R.

Post-order: L, R, N (e.g., deleting the tree structure).

In-order Traversal and BSTs

An in-order traversal of a BST visits the keys in ascending sorted order. This is a direct consequence of the BST invariant.

Python Generators and ‘yield‘

Implementing Traversal: The Challenge

We want to implement the ‘walk()‘ method to iterate over the tree (e.g., in a ‘for‘ loop).

Naive Approach: Building a List

1. Create an empty list.
2. Perform the in-order traversal recursively.
3. Append each element to the list.
4. Return the completed list.

The Goal: Lazy Evaluation

We want to generate the elements one by one, on demand.

Implementing Traversal: The Challenge

We want to implement the ‘walk()‘ method to iterate over the tree (e.g., in a ‘for‘ loop).

Naive Approach: Building a List

1. Create an empty list.
2. Perform the in-order traversal recursively.
3. Append each element to the list.
4. Return the completed list.

Drawbacks

- **Memory Overhead:** Requires $\mathcal{O}(N)$ extra space to store the entire sequence.
- **Latency (Not Lazy):** The user must wait for the entire traversal to finish before processing the first element.

Implementing Traversal: The Challenge

We want to implement the ‘walk()‘ method to iterate over the tree (e.g., in a ‘for‘ loop).

Naive Approach: Building a List

1. Create an empty list.
2. Perform the in-order traversal recursively.
3. Append each element to the list.
4. Return the completed list.

The Goal: Lazy Evaluation

We want to generate the elements one by one, on demand.

Introducing Python Generators and ‘yield‘

Generators

A generator is a special type of iterator in Python. A function containing the ‘yield‘ keyword automatically becomes a generator.

The ‘yield‘ Keyword

- ‘return‘ terminates the function and returns a value. State is lost.
- ‘yield‘ returns a value (produces an element) but **suspends** the function’s execution state (local variables, instruction pointer).
- When the generator is iterated again, execution resumes immediately after the ‘yield‘ statement.

Introducing Python Generators and ‘yield‘

Generators

A generator is a special type of iterator in Python. A function containing the ‘yield‘ keyword automatically becomes a generator.

The ‘yield‘ Keyword

- ‘return‘ terminates the function and returns a value. State is lost.
- ‘yield‘ returns a value (produces an element) but **suspends** the function’s execution state (local variables, instruction pointer).
- When the generator is iterated again, execution resumes immediately after the ‘yield‘ statement.

Introducing Python Generators and ‘yield‘

Generators

A generator is a special type of iterator in Python. A function containing the ‘yield‘ keyword automatically becomes a generator.

Listing 3: yield-example-1.py

```
# Using return
def return_example():
    return [1, 2, 3] # Function ends here

# Using yield
def yield_example():
    yield 1 # Function pauses here
    yield 2 # Resumes here
    yield 3
```

The Efficiency of ‘yield‘ (Lazy Evaluation)

‘yield‘ enables lazy evaluation, improving efficiency for sequences.

Listing 4: yield-example-2.py (Conceptual)

```
import time

# Using return (Eager)
def return_example():
    # ... (Simulates work, sleeps 1s per item) ...
    return result # Returns only after all work (3s) is done

# Using yield (Lazy)
def yield_example():
    for i in range(1, 4):
        time.sleep(1)  # Simulate work
        yield i # Returns immediately after this step's work (1s)
```

The Efficiency of ‘yield‘ (Lazy Evaluation)

‘yield‘ enables lazy evaluation, improving efficiency for sequences.

Observation

With ‘yield‘, the consumer (the ‘for‘ loop) can start processing the first element as soon as it’s ready, rather than waiting for the entire collection.

Delegating Generators: ‘yield from‘

When dealing with recursive structures like trees, we need a way for a generator to yield values produced by a recursive call (a sub-generator).

The ‘yield from‘ Keyword

‘yield from’ generator delegates execution to the sub-generator. It yields all values produced by the sub-generator until it completes, then resumes the current generator.

```
1 def subgenerator():
2     yield 'a'; yield 'b'
3
4 def yield_from_example():
5     yield 1
6     yield from subgenerator() # Yields 'a', 'b'
7     yield 2
8 # Output: 1, a, b, 2
```

BST Implementation: In-Order Traversal (Walk)

We can implement the in-order traversal efficiently using ‘yield‘ and ‘yield from‘.

Listing 5: TreeNode.walk (tree.py)

```
16      def walk(self):
17          # 1. Traverse Left Subtree (L)
18          if self.left is not None:
19              yield from self.left.walk()
20
21          # 2. Visit Node (N)
22          yield (self.key, self.value)
23
24          # 3. Traverse Right Subtree (R)
25          if self.right is not None:
26              yield from self.right.walk()
```

BST Implementation: In-Order Traversal (Walk)

We can implement the in-order traversal efficiently using ‘yield‘ and ‘yield from‘.

Analysis

This implementation is concise, clearly reflects the L-N-R structure, and is memory efficient ($\mathcal{O}(\mathrm{H})$ space for the recursion stack, not $\mathcal{O}(\mathrm{N})$ for storing results).

Advanced BST Operation: Deletion

The Challenge of Deletion

Deleting a node from a BST is complex because the operation must restructure the tree while maintaining the BST invariant.

The Strategy

We locate the node to delete (D). There are three cases based on the number of children D has.

The Recursive Structure of ‘delete‘

The ‘delete(key)‘ method must return the root of the (potentially modified) subtree. This allows the parent node to update its child pointer correctly.

The Challenge of Deletion

Deleting a node from a BST is complex because the operation must restructure the tree while maintaining the BST invariant.

The Strategy

We locate the node to delete (D). There are three cases based on the number of children D has.

The Recursive Structure of ‘delete‘

The ‘delete(key)‘ method must return the root of the (potentially modified) subtree. This allows the parent node to update its child pointer correctly.

The Challenge of Deletion

Deleting a node from a BST is complex because the operation must restructure the tree while maintaining the BST invariant.

The Strategy

We locate the node to delete (D). There are three cases based on the number of children D has.

The Recursive Structure of ‘delete‘

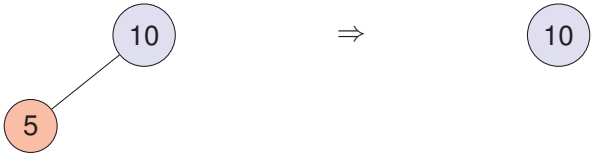
The ‘delete(key)‘ method must return the root of the (potentially modified) subtree. This allows the parent node to update its child pointer correctly.

```
# Example: Parent asks left child to delete and updates its pointer
self.left = self.left.delete(key)
```

Deletion Case 1: D is a Leaf (0 children)

Action

Simply remove the pointer from its parent to D. The ‘delete‘ method returns ‘None‘.

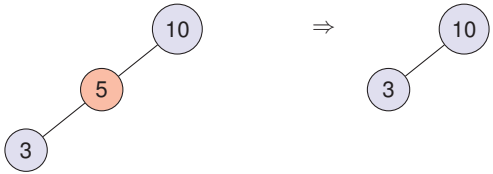



```
1 if self.left is None and self.right is None:
2     return None
```

Deletion Case 2: D has 1 child

Action

Replace D with its child. The parent of D now points directly to the child of D. The invariant is preserved.




```
1 elif self.left is None:
2     return self.right
3 elif self.right is None:
4     return self.left
```

Deletion Case 3: Node has 2 children (The Hard Case) I

If we delete the node, we create two disconnected subtrees. We need a replacement node (R) that maintains the invariant.

Requirements for the Replacement Node R

R must be greater than everything in the left subtree AND less than everything in the right subtree.

The Candidates

There are exactly two nodes that satisfy this:

1. **In-order Predecessor:** The largest key in the left subtree.
2. **In-order Successor:** The smallest key in the right subtree.

Deletion Case 3: Node has 2 children (The Hard Case) I

If we delete the node, we create two disconnected subtrees. We need a replacement node (R) that maintains the invariant.

Requirements for the Replacement Node R

R must be greater than everything in the left subtree AND less than everything in the right subtree.

The Candidates

There are exactly two nodes that satisfy this:

1. **In-order Predecessor:** The largest key in the left subtree.
2. **In-order Successor:** The smallest key in the right subtree.

Deletion Case 3: Node has 2 children (The Hard Case) II

The Algorithm (Using Predecessor)

1. Find the in-order predecessor (P).
2. Replace the data (key/value) of the node to be deleted (D) with the data of P.
3. Recursively delete the original node P from the left subtree.

Key Insight

The predecessor P is guaranteed to have at most 1 child. Therefore, the recursive deletion of P falls under Case 1 or Case 2, preventing infinite recursion.

Deletion Case 3: Node has 2 children (The Hard Case) II

The Algorithm (Using Predecessor)

1. Find the in-order predecessor (P).
2. Replace the data (key/value) of the node to be deleted (D) with the data of P.
3. Recursively delete the original node P from the left subtree.

Key Insight

The predecessor P is guaranteed to have at most 1 child. Therefore, the recursive deletion of P falls under Case 1 or Case 2, preventing infinite recursion.

BST Implementation: Finding the Predecessor

We need a helper function to find the maximum key in a subtree (the rightmost node).

Listing 6: TreeNode.maxkey (tree.py)

```
72      def maxkey(self):
73          if self.right is None:
74              return self.key
75          return self.right.maxkey()
```

BST Implementation: Deletion (Case 3 - Value Swapping) I

The strategy implemented in ‘tree.py‘ and ‘tree-fixed-question-response.py‘ uses value swapping (Hibbard Deletion).

Listing 7: TreeNode.delete (Case 3 - Value Swap)

```
1            else:
2                # 1. Find the maximum node in the left subtree (
                     Predecessor)
3                maxkey = self.left.maxkey()
4                maxnode = self.left.find(maxkey)
5
6                # 2. Replace the value of the current node with the
                     predecessor's
7                self.key = maxnode.key
```

BST Implementation: Deletion (Case 3 - Value Swapping) II

```
8                    self.value = maxnode.value
9
10                   # 3. Delete the predecessor node from the left
                        subtree
11                   self.left = self.left.delete(maxkey)
12                   return self
```

Analysis

This approach is common and robust. It avoids complex pointer manipulation by reducing Case 3 to a data update followed by a simpler deletion (Case 1 or 2).

Alternative Strategy: Deletion (Case 3 - Node Replacement) I

An alternative approach, shown in ‘tree-fixed.py‘, involves directly manipulating the pointers (Node Replacement).

Listing 8: TreeNode.delete (Case 3 - Node Replacement)

```
            else:
                # 1. Find the predecessor
                maxkey = self.left.maxkey()
                maxnode = self.left.find(maxkey)

                # 2. Remove the predecessor from the left subtree
                self.left = self.left.delete(maxkey)

```

Alternative Strategy: Deletion (Case 3 - Node Replacement) II

```
9                   # 3. Replace the current node with the predecessor
                        node
10                  #    (Attach the existing children to the
                        predecessor)
11                  maxnode.left = self.left
12                  maxnode.right = self.right
13
14                  # 4. Return the predecessor as the new root of the
                        subtree
15                  return maxnode
```

Alternative Strategy: Deletion (Case 3 - Node Replacement) III

Analysis

This approach is preferred if copying the ‘value‘ is expensive, but requires meticulous pointer management.

BST Analysis and Limitations

BST Complexity Analysis

The performance of all core BST operations (Insert, Lookup, Delete) is $\mathcal{O}(H)$.

Best Case / Average Case (Balanced Tree)

If the tree is balanced (e.g., random insertions), the height is logarithmic.

$$H \approx \log_2 N$$

Operations are $\mathcal{O}(\log N)$. This is very efficient.

Worst Case (Degenerate Tree)

If elements are inserted in sorted order, the tree degenerates into a linked list.

$$H = N$$

Operations degrade to $\mathcal{O}(N)$.

BST Complexity Analysis

The performance of all core BST operations (Insert, Lookup, Delete) is $\mathcal{O}(H)$.

Best Case / Average Case (Balanced Tree)

If the tree is balanced (e.g., random insertions), the height is logarithmic.

$$H \approx \log_2 N$$

Operations are $\mathcal{O}(\log N)$. This is very efficient.

Worst Case (Degenerate Tree)

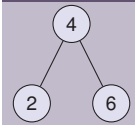
If elements are inserted in sorted order, the tree degenerates into a linked list.

$$H = N$$

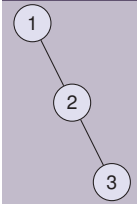
Operations degrade to $\mathcal{O}(N)$.

The Problem of Balance

Balanced ($\mathcal{O}(\log N)$)



Degenerate ($\mathcal{O}(N)$)



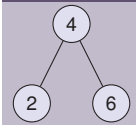
The Limitation

Standard BSTs do not guarantee balance. Performance depends heavily on insertion order.

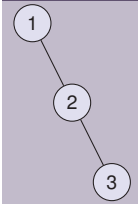
The Solution: Self-Balancing Trees

The Problem of Balance

Balanced ($\mathcal{O}(\log N)$)



Degenerate ($\mathcal{O}(N)$)



The Limitation

Standard BSTs do not guarantee balance. Performance depends heavily on insertion order.

The Solution: Self-Balancing Trees

The Problem of Balance

The Limitation

Standard BSTs do not guarantee balance. Performance depends heavily on insertion order.

The Solution: Self-Balancing Trees

To guarantee $\mathcal{O}(\log N)$ performance, specialised structures (e.g., AVL Trees, Red-Black Trees) automatically rebalance themselves during operations.

Hash Tables: Achieving Constant Time

The Quest for $\mathcal{O}(1)$

BSTs provide $\mathcal{O}(\log N)$ average time. Can we achieve $\mathcal{O}(1)$ lookups?

The Ideal: Direct Addressing

If keys are small integers (0 to $M - 1$), we use an array of size M. The key is the index.

- Insert(K, V): Array[K] = V. ($\mathcal{O}(1)$)
- Lookup(K): return Array[K]. ($\mathcal{O}(1)$)

The Limitations

- **Space:** Impractical if the universe of possible keys is huge (e.g., all strings).
- **Non-Integer Keys:** Needs adaptation for other data types.

The Quest for $\mathcal{O}(1)$

BSTs provide $\mathcal{O}(\log N)$ average time. Can we achieve $\mathcal{O}(1)$ lookups?

The Ideal: Direct Addressing

If keys are small integers (0 to $M - 1$), we use an array of size M. The key is the index.

- Insert(K, V): Array[K] = V. ($\mathcal{O}(1)$)
- Lookup(K): return Array[K]. ($\mathcal{O}(1)$)

The Limitations

- **Space:** Impractical if the universe of possible keys is huge (e.g., all strings).
- **Non-Integer Keys:** Needs adaptation for other data types.

The Hash Table Concept

Hash Tables generalise direct addressing when the number of stored keys N is much smaller than the universe of possible keys U.

The Strategy

1. Use an underlying array (buckets) of size M.
2. Use a **Hash Function** H to map keys from U into indices $\{0, \ldots, M-1\}$.

The Mapping Process

Typically: $\text{Index} = \text{hash}(K) \pmod{M}$.

The Hash Table Concept

Hash Tables generalise direct addressing when the number of stored keys N is much smaller than the universe of possible keys U.

The Strategy

1. Use an underlying array (buckets) of size M.
2. Use a **Hash Function** H to map keys from U into indices $\{0, \ldots, \mathrm{M}-1\}$.

The Mapping Process

Typically: Index $=$ hash(K) $(\mathrm{mod}\ \mathrm{M})$.

The Collision Problem

The Inevitability of Collisions

Different keys may map to the same index. This is a **collision**.

$$K_1 \neq K_2 \quad \text{but} \quad H(K_1) = H(K_2)$$

Requirement

A Hash Table must have a strategy for resolving collisions.

Collision Resolution Strategies

1. **Separate Chaining:** Store colliding elements in a list at the bucket.
2. **Open Addressing (Probing):** If a bucket is full, search for an empty slot elsewhere in the array. (Used by CPython).

The Collision Problem

The Inevitability of Collisions

Different keys may map to the same index. This is a **collision**.

$$K_1 \neq K_2 \quad \text{but} \quad H(K_1) = H(K_2)$$

Requirement

A Hash Table must have a strategy for resolving collisions.

Collision Resolution Strategies

1. **Separate Chaining:** Store colliding elements in a list at the bucket.
2. **Open Addressing (Probing):** If a bucket is full, search for an empty slot elsewhere in the array. (Used by CPython).

The Collision Problem

The Inevitability of Collisions

Different keys may map to the same index. This is a **collision**.

$$K_1 \neq K_2 \quad \text{but} \quad H(K_1) = H(K_2)$$

Requirement

A Hash Table must have a strategy for resolving collisions.

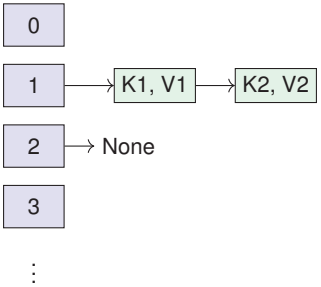
Collision Resolution Strategies

1. **Separate Chaining:** Store colliding elements in a list at the bucket.
2. **Open Addressing (Probing):** If a bucket is full, search for an empty slot elsewhere in the array. (Used by CPython).

Collision Resolution: Separate Chaining

The Strategy

Each bucket stores a pointer to a secondary data structure (typically a linked list) holding all entries that hash to that index.



Hash Table Implementation: Structure (hashtable.py)

Analysing the provided implementation using separate chaining.

Listing 9: HashTable Initialization

```
class HashTable:
    def __init__(self, element_list):
        # Initialise buckets (Example strategy: approx double the
            input size)
        self.bucket_count = len(element_list) * 2 - 1
        # Create a list of empty lists (The Chains)
        self.buckets = [[] for i in range(self.bucket_count)]
        # ... insert elements ...

    def get_index(self, key):
        # Use Python's built-in hash() and the modulo operator
        return hash(key) % self.bucket_count
```

Hash Table Implementation: Insert and Lookup I

Listing 10: HashTable Insert and Lookup

```
1     def insert(self, key, value):
2         index = self.get_index(key)
3         # Append the (key, value) tuple to the chain (list)
4         self.buckets[index].append( (key,value) )
5
6     def lookup(self, search_key):
7         index = self.get_index(search_key)
8         # Linear search within the chain
9         for key, value in self.buckets[index]:
10            if key == search_key:
11                return value
12        return None
```

Hash Table Implementation: Insert and Lookup II

Note

This simple implementation does not handle key updates (it allows duplicate keys) and lacks dynamic resizing.

Performance Analysis and Load Factor

Complexity Analysis (Separate Chaining)

The time complexity depends on the length of the chains.

- The **Load Factor** $\alpha = \mathrm{N}/\mathrm{M}$ (Entries / Buckets) is the average chain length.
- Assuming a good hash function, the expected time for lookup and insert is $\mathcal{O}(1 + \alpha)$.

Maintaining Performance: Rehashing

If we ensure α is kept constant (e.g., $\alpha < 1$), operations remain $\mathcal{O}(1)$ on average. This requires resizing (rehashing) the table when the load factor gets too high.

Amortisation

Rehashing takes $\mathcal{O}(\mathrm{N})$ time, but similar to dynamic arrays, the cost is amortised, resulting in $\mathcal{O}(1)$ average insertion time.

Performance Analysis and Load Factor

Complexity Analysis (Separate Chaining)

The time complexity depends on the length of the chains.

- The **Load Factor** $\alpha = \mathrm{N}/\mathrm{M}$ (Entries / Buckets) is the average chain length.
- Assuming a good hash function, the expected time for lookup and insert is $\mathcal{O}(1 + \alpha)$.

Maintaining Performance: Rehashing

If we ensure α is kept constant (e.g., $\alpha < 1$), operations remain $\mathcal{O}(1)$ on average. This requires resizing (rehashing) the table when the load factor gets too high.

Amortisation

Rehashing takes $\mathcal{O}(\mathrm{N})$ time, but similar to dynamic arrays, the cost is amortised, resulting in $\mathcal{O}(1)$ average insertion time.

Performance Analysis and Load Factor

Complexity Analysis (Separate Chaining)

The time complexity depends on the length of the chains.

- The **Load Factor** $\alpha = \mathrm{N}/\mathrm{M}$ (Entries / Buckets) is the average chain length.
- Assuming a good hash function, the expected time for lookup and insert is $\mathcal{O}(1 + \alpha)$.

Maintaining Performance: Rehashing

If we ensure α is kept constant (e.g., $\alpha < 1$), operations remain $\mathcal{O}(1)$ on average. This requires resizing (rehashing) the table when the load factor gets too high.

Amortisation

Rehashing takes $\mathcal{O}(\mathrm{N})$ time, but similar to dynamic arrays, the cost is amortised, resulting in $\mathcal{O}(1)$ average insertion time.

Python Dictionaries and Hashability I

Python's 'dict' and 'set' are highly optimised Hash Tables.

The Requirement of Hashability

To be used as a key in a 'dict' or element in a 'set', an object must be **hashable**.

Definition of Hashable

An object is hashable if its hash value never changes during its lifetime (it must be immutable).

Python Dictionaries and Hashability I

Python's 'dict' and 'set' are highly optimised Hash Tables.

The Requirement of Hashability

To be used as a key in a 'dict' or element in a 'set', an object must be **hashable**.

Definition of Hashable

An object is hashable if its hash value never changes during its lifetime (it must be immutable).

Python Dictionaries and Hashability II

Examples

Immutable types (‘int‘, ‘str‘, ‘tuple‘) are hashable. Mutable types (‘list‘, ‘dict‘, ‘set‘) are **not** hashable.

```
# This will fail:
my_dict = { [1, 2]: "value" }
# TypeError: unhashable type: 'list'
```

Summary and Conclusion

Summary of Data Structures and Performance

Data Structure	Insert (Avg)	Delete (Avg)	Lookup (Avg)	Worst Case Lookup
Unsorted Array/List	$\mathcal{O}(1)^{\mathcal{A}}$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Sorted Array/List	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$
Binary Search Tree (BST)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
Balanced BST (e.g., AVL)	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$
Hash Table (Dict/Set)	$\mathcal{O}(1)^{\mathcal{A}}$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(N)$

$\mathcal{A}$ = Amortised

Key Takeaways I

Binary Search Trees

- Hierarchical structure maintaining the BST invariant for efficient search ($\mathcal{O}(\mathrm{H})$).
- Deletion is complex, especially the 2-child case (requires predecessor/successor).
- Performance degrades to $\mathcal{O}(\mathrm{N})$ if the tree becomes unbalanced.
- Maintains sorted order (efficient in-order traversal).

Python Mechanisms

- Generators (‘yield‘, ‘yield from‘) enable lazy evaluation and memory-efficient iteration over complex structures.

Key Takeaways II

Hash Tables

- Utilise hash functions for $\mathcal{O}(1)$ average time operations.
- Require collision resolution strategies (e.g., Separate Chaining).
- Performance depends on a good hash function and maintaining a low load factor via rehashing.